BÁO CÁO ĐỒ ÁN
HÀNH TRÌNH AN NHIÊN

Xây dựng ứng dụng di động hỗ trợ hành hương và tụng kinh

Chương 2 Cơ sở lý thuyết và công nghệ sử dụng
Chương 3 Phân tích và thiết kế hệ thống
Chương 4 Hiện thực hệ thống kiểm thử và triển khai

Digital Pilgrimage App
Tháng 10 năm 2026

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ SỬ DỤNG

2.1. Cơ sở lựa chọn công nghệ và định hướng xây dựng hệ thống

Ứng dụng **Hành Trình An Nhiên – Digital Pilgrimage App** được xây dựng nhằm kết hợp các hoạt động tra cứu thông tin Phật giáo, đọc và tụng kinh, quản lý lịch cá nhân và ghi nhận hành trình viếng chùa trên cùng một nền tảng di động. Đặc điểm của bài toán là có sự kết hợp giữa nội dung đọc tương đối ổn định, dữ liệu cá nhân cần bảo vệ và các tương tác với phần cứng như NFC, camera, vị trí và sinh trắc học.

Vì vậy, việc lựa chọn công nghệ không chỉ dựa trên khả năng xây dựng giao diện mà còn cần đáp ứng khả năng kết nối dịch vụ đám mây, quản lý dữ liệu lâu dài, tích hợp phần cứng và triển khai tự động. Hệ thống được tổ chức theo mô hình client–server, trong đó ứng dụng Flutter đảm nhiệm tương tác với người dùng, còn backend xử lý các quy tắc nghiệp vụ và truy cập cơ sở dữ liệu.

Các tiêu chí lựa chọn công nghệ gồm:

1. **Khả năng phát triển đa nền tảng:** tận dụng phần lớn mã nguồn cho Android và iOS.
2. **Khả năng tích hợp phần cứng:** hỗ trợ NFC, camera, GPS và xác thực sinh trắc học.
3. **Khả năng bảo vệ dữ liệu cá nhân:** kiểm tra danh tính, quyền sở hữu và trạng thái tài khoản ở backend.
4. **Khả năng triển khai trong điều kiện đồ án:** sử dụng các dịch vụ phù hợp với nguồn lực và chi phí của nhóm.
5. **Khả năng kiểm thử và bảo trì:** tách biệt giao diện, dịch vụ, mô hình dữ liệu và xử lý nghiệp vụ.
6. **Khả năng mở rộng:** bổ sung dữ liệu chùa, hình ảnh, kinh sách và chức năng mà không phải thay đổi toàn bộ hệ thống.

Bảng 2.1. Tổng hợp các công nghệ chính

Thành phần	Công nghệ hoặc thư viện	Phiên bản/cấu hình ghi nhận	Vai trò
Framework di động	Flutter	`3.47.4` trong CI	Xây dựng giao diện Android/iOS
Ngôn ngữ di động	Dart	Ràng buộc SDK `^3.13.3`	Xử lý giao diện và logic phía client
Thành phần Android native	Kotlin	Theo cấu hình Android của dự án	Kết nối Flutter với nền tảng Android
Công cụ biên dịch Android	Java, Gradle Kotlin DSL	CI sử dụng Java 17	Biên dịch và đóng gói Android
Gọi API	`http`	`^1.6.0`	Gửi và nhận dữ liệu HTTP
Lưu cấu hình cục bộ	`shared_preferences`	`^2.5.6`	Lưu tùy chọn và một số trạng thái ứng dụng
NFC	`nfc_manager`	`^4.2.1`	Giao tiếp với NFC
Dữ liệu NDEF	`nfc_manager_ndef`	`^1.1.0`	Xử lý bản ghi NDEF

Thành phần	Công nghệ hoặc thư viện	Phiên bản/cấu hình ghi nhận	Vai trò
Quét QR	`mobile_scanner`	`^7.4.2`	Đọc mã bằng camera
Sinh trắc học	`local_auth`	`^3.0.2`	Xác thực qua cơ chế của thiết bị
Chọn ảnh	`image_picker`	`^1.2.4`	Chọn ảnh đại diện
Xử lý kiểu nội dung HTTP	`http_parser`	`^4.1.2`	Hỗ trợ dữ liệu tải lên
Mở liên kết ngoài	`url_launcher`	`^6.3.3`	Mở bản đồ và nhóm cộng đồng
Firebase nền tảng	`firebase_core`	`^4.15.2`	Khởi tạo Firebase
Xác thực Firebase	`firebase_auth`	`^6.7.1`	Email/password và phiên Firebase
Đăng nhập Google	`google_sign_in`	`^7.2.0`	Đăng nhập bằng tài khoản Google
Định vị	`geolocator`	`^14.1.2`	Lấy vị trí và tính khoảng cách
Thông báo cục bộ	`flutter_local_notifications`	`^22.3.2`	Nhắc lịch trên điện thoại
Múi giờ	`timezone`	`^0.11.2`	Hỗ trợ lập lịch thông báo
Runtime backend	Node.js	`22`, Docker Alpine	Chạy dịch vụ backend
Framework backend	Express	`^5.2.1`	Xây dựng REST API
ODM	Mongoose	`^9.10.2`	Mô hình hóa và truy cập MongoDB
Xác thực phía máy chủ	Firebase Admin SDK	`^14.5.0`	Xác minh Firebase ID token
Token legacy	`jsonwebtoken`	`^9.0.3`	Phát hành và kiểm tra JWT
Băm mật khẩu legacy	`bcryptjs`	`^3.0.3`	Băm và đối chiếu mật khẩu
Xử lý tải tệp	`multer`	`^2.4.0`	Tiếp nhận dữ liệu multipart
Cấu hình môi trường	`dotenv`	`^18.0.3`	Nạp biến môi trường
Cấu hình CORS	`cors`	`^2.8.6`	Thiết lập truy cập liên nguồn
Hỗ trợ phát triển backend	`nodemon`	`^3.1.14`	Tự khởi động lại khi sửa mã
Database kiểm thử	MongoDB	Image `mongo:7`	Chạy kiểm thử với replica set
Cache và rate limit	Redis qua Upstash REST	Cấu hình môi trường	Giảm truy vấn và hạn chế spam
Lưu ảnh đại diện	Cloudinary	Dịch vụ bên ngoài	Lưu trữ ảnh tải lên
CI/CD	GitHub Actions, GHCR	Theo workflow trong repository	Kiểm thử và xuất bản Docker image
Hosting backend	Render	Docker Web Service	Chạy API trên Internet

Lưu ý về phiên bản: ký hiệu ^ trong các tệp khai báo là khoảng phiên bản được cho phép, không phải lúc nào cũng là phiên bản chính xác đang được cài đặt. Khi cần tái lập môi trường, phải kết hợp các tệp khóa như `pubspec.lock` và `package-lock.json`.

2.2. Công nghệ phía ứng dụng di động

2.2.1. Ngôn ngữ Dart

Dart là ngôn ngữ sử dụng để xây dựng phần lớn mã nguồn phía ứng dụng. Trong đồ án, Dart được dùng để định nghĩa các mô hình dữ liệu, xây dựng giao diện, gọi API, quản lý trạng thái và xử lý những tương tác của người dùng.

Một đặc điểm quan trọng của Dart đối với hệ thống là cơ chế xử lý bất đồng bộ thông qua `Future`, `async` và `await`. Các tác vụ như đăng nhập, tải danh sách chùa, lấy vị trí hoặc đọc dữ liệu từ backend không thể hoàn thành tức thời. Việc sử dụng bất đồng bộ giúp ứng dụng hiển thị trạng thái đang tải và tiếp tục phản hồi thay vì chặn toàn bộ giao diện.

Dart còn cung cấp null safety để phân biệt giá trị bắt buộc với giá trị có thể không tồn tại. Điều này phù hợp với dữ liệu thực tế của đồ án: một ngôi chùa có thể chưa có tọa độ, một nhân vật Phật giáo có thể chưa có ảnh và một câu Phật ngôn có thể chưa có thông tin tác giả.

Việc biểu diễn các trường tùy chọn một cách rõ ràng giúp ứng dụng chủ động sử dụng nội dung thay thế, đồng thời giảm nguy cơ lỗi khi truy cập dữ liệu rỗng.

2.2.2. Flutter và cơ chế xây dựng giao diện

Flutter tổ chức giao diện dưới dạng cây widget. Các thành phần như danh sách chùa, thẻ Phật ngôn, lịch, biểu mẫu đăng nhập và trang đọc kinh được cấu thành từ những widget nhỏ hơn.

Trong đồ án, cách tổ chức này mang lại ba lợi ích chính:

- Có thể tái sử dụng các thành phần chung giữa nhiều màn hình.
- Dễ tách phần trình bày khỏi phần gọi API.
- Có thể kiểm thử một thành phần giao diện bằng widget test mà không cần chạy toàn bộ ứng dụng trên điện thoại.

Ứng dụng sử dụng phong cách giao diện chủ đạo gồm nền sáng, màu nâu, màu vàng và các khối nội dung bo góc. Những lựa chọn này phục vụ mục tiêu trình bày nội dung dễ đọc và tạo tính nhất quán giữa trang chủ, danh sách chùa, kinh sách và tài khoản.

Flutter hỗ trợ phát triển đa nền tảng nhưng không loại bỏ hoàn toàn cấu hình riêng của từng hệ điều hành. Android và iOS vẫn cần khai báo quyền, thiết lập đăng nhập Google, cấu hình ứng dụng Firebase và xử lý giới hạn của các plugin phần cứng.

2.2.3. Quản lý trạng thái bằng các thành phần có sẵn của Flutter

Dự án sử dụng cách quản lý trạng thái dựa trên `StatefulWidget`, `setState` và các cơ chế thông báo thay đổi như Notifier/Listenable ở những thành phần phù hợp. Tầng Services được dùng để tách các thao tác dữ liệu khỏi giao diện.

Ví dụ, thẻ Phật ngôn trên trang chủ có các trạng thái:

1. Đang tải dữ liệu.

2. Tải thành công và có câu kệ.
3. Không có dữ liệu.
4. Gặp lỗi kết nối.
5. Đang tải một câu khác.

Nếu chỉ hiển thị văn bản mà không quản lý các trạng thái này, người dùng sẽ khó phân biệt giữa lỗi mạng và trường hợp hệ thống chưa có dữ liệu. Bởi vậy, trạng thái của giao diện là một phần của thiết kế nghiệp vụ chứ không chỉ là chi tiết trình bày.

Cách tiếp cận hiện tại phù hợp với quy mô đồ án, đồng thời giảm số lượng thư viện phụ thuộc. Khi số lượng màn hình và trạng thái chia sẻ tăng đáng kể, nhóm có thể đánh giá lại nhu cầu sử dụng một giải pháp quản lý trạng thái tập trung.

2.2.4. Tầng Services và giao tiếp REST API

Các lớp Services đóng vai trò trung gian giữa giao diện và backend. Thay vì viết trực tiếp mã HTTP trong từng widget, hệ thống tổ chức các dịch vụ theo nhóm nghiệp vụ như xác thực, chùa, kinh sách, Phật ngôn, NFC, hành hương và lịch cá nhân.

Một tác vụ gọi API thường gồm các bước:

1. Xây dựng địa chỉ endpoint.
2. Gắn tham số truy vấn hoặc nội dung yêu cầu.
3. Bổ sung token nếu endpoint yêu cầu đăng nhập.
4. Gửi yêu cầu với thời gian chờ phù hợp.
5. Kiểm tra mã trạng thái HTTP.
6. Giải mã JSON.
7. Chuyển dữ liệu thành đối tượng Dart.
8. Trả kết quả hoặc lỗi về giao diện.

Việc tập trung logic này trong Services giúp giảm lặp mã và tạo điều kiện cho kiểm thử bằng HTTP client giả lập.

2.2.5. Cấu hình môi trường bằng `--dart-define`

Ứng dụng cần các thông số khác nhau tùy môi trường chạy, chẳng hạn địa chỉ API và cấu hình Firebase. Dự án sử dụng giá trị được truyền khi biên dịch thông qua `--dart-define` hoặc `--dart-define-from-file`.

Ví dụ:

```
flutter build apk --release --dart-define-from-file=firebase-config.json
```

Cơ chế này cho phép dùng cùng một mã nguồn để tạo các bản ứng dụng kết nối đến môi trường khác nhau. Chẳng hạn, bản thử nghiệm có thể gọi backend nội bộ, trong khi bản phát hành gọi API trên Render.

Tuy nhiên, cấu hình được nhúng vào ứng dụng không được xem là kho bí mật. Các thông tin như private key của Firebase Admin, mật khẩu MongoDB, Cloudinary API secret hoặc Upstash REST token phải được giữ ở backend.

2.2.6. Tích hợp NFC và dữ liệu NDEF

NFC là công nghệ giao tiếp ở khoảng cách gần, được ứng dụng sử dụng để đọc thẻ tại chùa. Trong thiết kế của đồ án, thao tác chạm thẻ đóng vai trò khởi đầu cho quy trình check-in.

Cần phân biệt hai loại thông tin:

- **UID:** định danh của thẻ mà ứng dụng và backend sử dụng để tra cứu đăng ký.
- **NDEF:** định dạng chứa các bản ghi dữ liệu như văn bản hoặc URI trên thẻ.

`nfc_manager` phụ trách tương tác với khả năng NFC của nền tảng. `nfc_manager_ndef` hỗ trợ xử lý nội dung NDEF khi loại thẻ và thiết bị cho phép.

Dữ liệu đọc được từ thẻ không được xem là căn cứ duy nhất để cấp tem. Backend phải kiểm tra thẻ đã được đăng ký, còn hoạt động và thuộc về đúng ngôi chùa. Thiết kế này giúp tránh trường hợp client tự chọn một chùa rồi yêu cầu máy chủ cấp tem tùy ý.

Tuy nhiên, UID không phải chữ ký mật mã. Cơ chế hiện tại chưa đủ để khẳng định chống sao chép thẻ hoặc chứng minh tuyệt đối người dùng có mặt tại chùa. Đây là giới hạn cần được trình bày rõ trong báo cáo.

2.2.7. Camera và quét mã QR

Thư viện `mobile_scanner` cung cấp khả năng nhận diện mã bằng camera. Chức năng này tạo thêm phương thức tiếp nhận thông tin bên cạnh thao tác nhập thủ công và NFC.

Quét QR có lợi thế là nhiều thiết bị có camera nhưng không hỗ trợ NFC. Tuy nhiên, quét được một mã không đồng nghĩa với việc được phép cấp tem. Nếu QR được sử dụng trong một nghiệp vụ có quyền hạn, dữ liệu từ mã vẫn phải đi qua bước kiểm tra ở backend.

Ứng dụng cần xử lý các tình huống như người dùng từ chối quyền camera, mã không đúng định dạng hoặc camera đọc cùng một mã nhiều lần liên tiếp.

2.2.8. Sinh trắc học và thành phần Android native

Chức năng sinh trắc học được triển khai thông qua `local_auth`. Khi sử dụng, ứng dụng gọi cơ chế xác thực của hệ điều hành, còn việc đối chiếu vân tay hoặc khuôn mặt do thiết bị thực hiện.

Ứng dụng không trực tiếp thu thập ảnh vân tay và không lưu mẫu sinh trắc học vào MongoDB. Kết quả mà ứng dụng nhận được là trạng thái xác thực thành công, thất bại hoặc lỗi.

Trong đồ án, sinh trắc học là lớp bảo vệ quyền truy cập giao diện hoặc phiên sử dụng trên thiết bị. Nó không thay thế việc backend xác minh token của người dùng.

Ở Android, cấu hình Activity phải tương thích với yêu cầu của plugin. Kotlin đóng vai trò hỗ trợ tích hợp nền tảng, trong khi phần lớn giao diện và nghiệp vụ vẫn được viết bằng Dart.

2.2.9. Định vị GPS và tìm chùa gần

Thư viện `geolocator` được sử dụng để lấy vị trí hiện tại và tính khoảng cách giữa người dùng với chùa.

Quy trình xử lý bao gồm:

- Kiểm tra dịch vụ vị trí đã bật hay chưa.

- Kiểm tra quyền truy cập vị trí.
- Xin quyền khi cần.
- Xử lý trường hợp bị từ chối hoặc bị chặn vĩnh viễn.
- Lấy tọa độ hiện tại.
- Tính khoảng cách đến từng chùa có tọa độ hợp lệ.
- Sắp xếp danh sách theo khoảng cách.

Khoảng cách được tính từ tọa độ không phải khoảng cách đường đi thực tế. Chức năng chỉ đường được tách riêng bằng cách mở ứng dụng hoặc trang bản đồ bên ngoài.

Các chùa không có tọa độ hợp lệ không làm hỏng toàn bộ danh sách; chúng được xử lý như những phần tử chưa xác định được khoảng cách.

2.2.10. Thông báo cục bộ và xử lý thời gian

Ứng dụng sử dụng `flutter_local_notifications` và `timezone` để hỗ trợ nhắc lịch trên điện thoại.

Cần phân biệt:

- **Thông báo cục bộ:** ứng dụng lên lịch trên thiết bị.
- **Push notification:** máy chủ gửi thông báo từ xa qua một hạ tầng chuyển phát.

Phần đã triển khai trong đồ án là thông báo cục bộ. Việc sử dụng Firebase Authentication không đồng nghĩa hệ thống đã triển khai Firebase Cloud Messaging.

Bộ lập kế hoạch nhắc lịch tổng hợp các mốc cần nhắc, loại bỏ thời điểm đã qua, sắp xếp theo thời gian và giới hạn số lượng đang chờ. Mã nguồn hiện giới hạn tối đa 60 mục trong kế hoạch, đồng thời cập nhật lại khi ứng dụng mở hoặc dữ liệu lịch thay đổi.

2.3. Công nghệ backend và cơ sở dữ liệu

2.3.1. Node.js 22

Node.js là môi trường thực thi JavaScript phía máy chủ. Backend của đồ án sử dụng Node.js 22 và được đóng gói với image Docker Alpine.

Node.js phù hợp với các tác vụ của ứng dụng như tiếp nhận HTTP request, truy vấn cơ sở dữ liệu, xác minh danh tính và gọi dịch vụ bên ngoài. Đây chủ yếu là các tác vụ I/O, trong đó thời gian chờ phản hồi thường lớn hơn thời gian tính toán thuần túy.

Việc sử dụng cùng hệ sinh thái JavaScript cho backend giúp quá trình phát triển thuận tiện, đồng thời cho phép sử dụng các API có sẵn như `fetch`, `FormData` và `node:crypto`.

2.3.2. Express và kiến trúc phân lớp

Express được sử dụng để xây dựng REST API. Các thành phần được tổ chức theo chuỗi xử lý:

```
HTTP Request
    → Route
    → Middleware
    → Controller
    → Model hoặc dịch vụ bên ngoài
    → HTTP Response
```

Vai trò của từng lớp:

Lớp	Trách nhiệm
Routes	Khai báo endpoint, HTTP method và chuỗi xử lý
Middleware	Xác thực, kiểm tra dữ liệu, cache, giới hạn request
Controllers	Điều phối nghiệp vụ và xây dựng phản hồi
Models	Khai báo cấu trúc, ràng buộc và chỉ mục dữ liệu
Config	Khởi tạo kết nối và cấu hình dịch vụ
Utils	Chứa các hàm hỗ trợ dùng chung

Cách tổ chức này giúp tránh tập trung toàn bộ mã nguồn vào một tệp duy nhất. Một thay đổi trong xử lý avatar không cần làm thay đổi logic của check-in, và thay đổi schema lịch cá nhân không nhất thiết ảnh hưởng đến danh sách kinh sách.

2.3.3. MongoDB và mô hình dữ liệu document

MongoDB lưu dữ liệu dưới dạng document. Kiểu dữ liệu này phù hợp với các thực thể có cấu trúc linh hoạt như chùa, nhân vật Phật giáo và kinh sách.

Ví dụ, một document chùa có thể chứa:

- Tên và slug.
- Địa chỉ được chia thành nhiều thành phần.
- Tọa độ.
- Ảnh đại diện và mảng ảnh.
- Giới thiệu, lịch sử.
- Giờ mở cửa.
- Thông tin liên hệ.

Các thành phần gắn chặt với một chùa được nhúng trong document, trong khi những quan hệ độc lập như người dùng, lần viếng và tem được tách thành các collection riêng.

Tên database được điều khiển bằng cấu hình môi trường. Vì vậy, tên `hanh_huong_db` trong đề cương cần được đối chiếu với cấu hình triển khai nếu đưa vào phần mô tả môi trường production.

2.3.4. Mongoose ODM

Mongoose là lớp ánh xạ giữa đối tượng JavaScript và document MongoDB. Các schema giúp định nghĩa:

- Kiểu dữ liệu.
- Giá trị mặc định.
- Trường bắt buộc.
- Độ dài tối đa.
- Tập giá trị được phép.
- Tham chiếu giữa các collection.

- Chỉ mục phục vụ truy vấn.

Ví dụ, trạng thái tài khoản chỉ nhận `active` hoặc `blocked`; danh mục nhân vật chỉ nhận các nhóm đã định nghĩa; tọa độ phải nằm trong khoảng hợp lệ.

Mongoose cũng hỗ trợ transaction thông qua session. Trong đồ án, transaction có ý nghĩa đặc biệt đối với quy trình tạo lần viếng và cấp tem, vì các thao tác này cần được xử lý nhất quán.

2.3.5. Redis và chiến lược cache

Redis được tích hợp qua giao diện REST của Upstash. Trong trạng thái mã nguồn đã kiểm tra, Redis có hai vai trò chính:

1. Cache phản hồi của các API đọc phù hợp.
2. Giới hạn tần suất request.

Đối với cache, hệ thống sử dụng khóa được tạo từ địa chỉ yêu cầu. Khi có dữ liệu hợp lệ trong cache, backend có thể trả kết quả mà không phải truy vấn MongoDB lại.

Cache hiện được áp dụng có điều kiện. Middleware bỏ qua yêu cầu không phải GET và yêu cầu có header xác thực. Quy tắc này giúp tránh đưa dữ liệu cá nhân vào một vùng cache dùng chung.

Dữ liệu cache có thời hạn sống. Khi hết hạn, backend đọc lại nguồn dữ liệu chính và có thể ghi cache mới.

Nếu cache gặp lỗi, hệ thống tiếp tục xử lý bằng MongoDB. Tuy nhiên, rate limit có cách xử lý khác: khi cơ chế giới hạn đã được bật nhưng Redis không hoạt động, middleware trả lỗi tạm thời thay vì âm thầm bỏ qua lớp chống spam.

2.3.6. Cloudinary và quản lý ảnh đại diện

Ảnh đại diện là dữ liệu được người dùng tạo trong quá trình sử dụng. Nếu lưu trên ổ đĩa tạm của một môi trường triển khai, ảnh có nguy cơ không còn sau khi dịch vụ được thay thế hoặc triển khai lại.

Cloudinary được sử dụng để tách lưu trữ ảnh đại diện khỏi vòng đời của container backend. Backend tiếp nhận ảnh, xử lý yêu cầu tải lên, gọi Cloudinary và lưu địa chỉ ảnh vào hồ sơ người dùng.

Dự án tích hợp bằng các API có sẵn của Node.js, gồm `fetch`, `FormData` và `node:crypto`. SHA-256 được sử dụng trong việc tạo chữ ký cho yêu cầu tương ứng; đây không phải thao tác mã hóa toàn bộ nội dung ảnh.

Cần phân biệt ảnh avatar tải lên Cloudinary với bộ ảnh chùa và kinh sách nằm trong dữ liệu, thư mục ảnh của dự án. Không nên mô tả toàn bộ ảnh của hệ thống đều đã được chuyển lên Cloudinary.

2.4. Công nghệ xác thực và bảo mật

2.4.1. Firebase Authentication

Firebase Authentication phụ trách xác minh danh tính qua hai phương thức chính:

- Email và mật khẩu.
- Google Sign-In.

Đối với đăng ký email, việc tạo tài khoản chưa phải điều kiện cuối cùng để được truy cập nghiệp vụ. Backend tiếp tục yêu cầu email đã được xác minh.

Đối với Google Sign-In, ứng dụng sử dụng thông tin xác thực từ Google để tạo phiên Firebase, sau đó gửi Firebase ID token đến backend. Backend không sử dụng trực tiếp một email do client tự khai báo để xác nhận danh tính.

2.4.2. Firebase Admin SDK và kiểm tra token

Firebase Admin SDK thực hiện kiểm tra ID token phía máy chủ. Mã nguồn còn kiểm tra trạng thái thu hồi và tài khoản bị vô hiệu hóa thông qua lời gọi xác minh có kiểm tra revocation.

Sau bước xác minh token, hệ thống kiểm tra thêm:

- `email_verified` phải bằng `true`.
- Email phải có kiểu dữ liệu hợp lệ.
- Nhà cung cấp đăng nhập thuộc nhóm được hỗ trợ.
- Người dùng ứng dụng phải tồn tại trong MongoDB.
- Trạng thái người dùng phải là `active`.

Việc kiểm tra đồng thời danh tính Firebase và trạng thái tài khoản nội bộ cho phép hệ thống tách hai khái niệm: "đã xác thực danh tính" và "được phép sử dụng nghiệp vụ".

2.4.3. Ý nghĩa của tiền tố `firebase:`

Header xác thực của ứng dụng có thể mang dạng:

```
Authorization: Bearer firebase:<Firebase-ID-Token>
```

Tiền tố `firebase:` là quy ước riêng của backend nhằm phân biệt token Firebase với token JWT legacy. Nó không phải một phần của chuẩn Firebase ID token và không phải cơ chế mã hóa.

Middleware loại bỏ tiền tố trước khi chuyển token cho Firebase Admin SDK. Vì vậy, độ an toàn đến từ bước xác minh chữ ký, claims và trạng thái tài khoản, không đến từ bản thân tiền tố.

2.4.4. JWT và bcryptjs trong cơ chế legacy

Hệ thống vẫn có cơ chế tương thích với tài khoản cũ. Mật khẩu legacy được băm bằng bcryptjs, còn JWT biểu diễn phiên truy cập.

Băm mật khẩu là phép biến đổi một chiều. Hệ thống không cần giải mã mật khẩu đã lưu mà sử dụng hàm so sánh để kiểm tra mật khẩu người dùng nhập.

Trường `tokenVersion` được sử dụng để vô hiệu hóa token cũ sau những thay đổi có liên quan. Middleware đối chiếu phiên bản trong token với phiên bản hiện tại của tài khoản; nếu không khớp, yêu cầu bị từ chối.

Cơ chế Firebase và legacy không được ghép tự động chỉ vì trùng email. Mã nguồn xử lý xung đột này bằng phản hồi phù hợp, tránh cấp quyền truy cập lịch sử của tài khoản cũ cho một danh tính mới chưa được liên kết rõ ràng.

2.4.5. Ký số ứng dụng Android

APK release cần được ký bằng khóa của nhà phát triển. Chữ ký giúp hệ điều hành nhận diện nguồn cập nhật và kiểm tra tính toàn vẹn của gói cài đặt.

Trong quá trình triển khai đồ án, khóa release đã được chuẩn bị và fingerprint được cấu hình phục vụ Google Sign-In. Các đặc điểm cụ thể như PKCS12 và RSA 3072 thuộc hồ sơ khóa đã tạo; khi trình bày như bằng chứng kỹ thuật, nên kèm kết quả kiểm tra khóa hoặc APK.

SHA-1 và SHA-256 trong Firebase là fingerprint của chứng chỉ ký. Chúng giúp nhận diện cấu hình ứng dụng tương ứng, không phải mật khẩu và cũng không phải thuật toán mã hóa dữ liệu MongoDB.

Một bản APK được biên dịch bằng cùng application ID nhưng dùng khóa khác có thể không cập nhật đè bản đang cài. Do đó, bảo quản keystore là yêu cầu quan trọng của quy trình phát hành.

2.5. Công nghệ kiểm thử, đóng gói và triển khai

2.5.1. Docker và Docker Compose

Docker đóng gói backend cùng môi trường chạy để giảm sự khác biệt giữa các máy. Dockerfile hiện sử dụng Node.js 22 Alpine, cài dependency bằng `npm ci --omit=dev` và chạy bằng tài khoản `node` thay vì root.

Docker Compose được sử dụng để khởi tạo đồng thời các thành phần phục vụ kiểm thử. Đặc biệt, MongoDB kiểm thử được cấu hình replica set để hỗ trợ transaction của quy trình check-in.

2.5.2. GitHub Actions

GitHub Actions tự động thực hiện kiểm tra khi có pull request hoặc cập nhật vào nhánh chính. Các bước phân tích mã và kiểm thử tạo thành điều kiện đánh giá trước khi tích hợp.

Giá trị của CI là cung cấp kết quả tái lập được trên môi trường tương đối độc lập với máy cá nhân. Tuy nhiên, CI không thay thế kiểm thử phần cứng thực tế, chẳng hạn đưa thẻ NFC đến điện thoại hoặc xác minh camera trên nhiều thiết bị.

2.5.3. GHCR và Render

GHCR lưu trữ Docker image của backend. Workflow CD tạo image theo commit đã vượt qua CI và gắn tag chứa SHA để truy vết.

Render là môi trường chạy API thực tế. Cấu hình repository sử dụng Docker và điều kiện tự triển khai khi các kiểm tra đạt.

Hai quy trình này cần được mô tả riêng: workflow GHCR xuất bản image, trong khi cấu hình Render hiện chỉ ra quy trình build từ Dockerfile của repository. Không nên khẳng định Render đang kéo image từ GHCR nếu chưa có cấu hình tương ứng.

2.5.4. Đóng gói iOS bằng môi trường macOS từ xa

Flutter vẫn cần công cụ biên dịch của Apple để tạo bản iOS. Vì người thực hiện không có máy macOS cá nhân, dự án sử dụng workflow trên môi trường macOS từ xa để tạo bản build chưa ký.

Gói cài đặt sau đó được xử lý bằng công cụ sideload trên máy người dùng. Người thực hiện đã xác nhận ứng dụng chạy thành công trên iOS.

Dù vậy, "đã chạy được" chưa đồng nghĩa tất cả tính năng đã tương đương Android. Đặc biệt, cấu hình sideload hiện tại có giới hạn đối với NFC và cần kiểm thử riêng các chức năng đăng nhập, thông báo, vị trí và xử lý vòng đời ứng dụng.

CHƯƠNG 3. PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Khảo sát bài toán và xác định phạm vi

Các hoạt động hành hương và tụng kinh thường sử dụng nhiều nguồn thông tin riêng biệt: bản đồ để tìm chùa, tài liệu để đọc kinh, lịch để theo dõi ngày lễ và các phương tiện cá nhân để lưu lại kỷ niệm.

Sự phân tán này tạo ra khó khăn trong việc theo dõi một hành trình thống nhất. Người dùng có thể nhớ tên một ngôi chùa nhưng không có lịch sử viếng thăm; có thể biết ngày lễ âm lịch nhưng chưa có công cụ nhắc lịch; hoặc có tài liệu kinh nhưng chưa thuận tiện đọc trong thời gian dài.

Ứng dụng giải quyết bài toán bằng cách tích hợp các chức năng trên trong một hệ thống dùng chung tài khoản và dữ liệu. Phạm vi không chỉ gồm tra cứu mà còn có nghiệp vụ ghi nhận check-in và cấp tem.

Bộ dữ liệu ban đầu dự kiến quản lý 21 ngôi chùa. Trong quá trình phát triển, dữ liệu cục bộ được mở rộng thành 27 chùa. Tại thời điểm lập báo cáo, dữ liệu mới đang ở trạng thái chờ kiểm tra trước khi seed, vì vậy không được đồng nhất số lượng dữ liệu nguồn với dữ liệu đang chạy trên production.

3.2. Xác định tác nhân

Bảng 3.1. Các tác nhân của hệ thống

Tác nhân	Vai trò
Khách chưa đăng nhập	Tra cứu các nội dung công khai
Người dùng đã đăng nhập	Quản lý hồ sơ, check-in, xem tem và lịch sử, quản lý lịch cá nhân
Người vận hành	Quản lý cấu hình triển khai, dữ liệu nguồn và các tác vụ quản trị được hỗ trợ
Firebase Authentication	Xác minh danh tính
Cloudinary	Tiếp nhận và lưu ảnh đại diện
Redis	Cache và giới hạn request
Hệ điều hành di động	Cấp quyền và cung cấp NFC, camera, sinh trắc học, vị trí, thông báo

Model người dùng có trường vai trò `user` và `admin`. Tuy nhiên, sự tồn tại của trường này không tự chứng minh hệ thống đã có một trang quản trị hoàn chỉnh. Các chức năng quản trị cần được mô tả theo endpoint và giao diện thực sự đã triển khai.

3.3. Phân tích yêu cầu chức năng

3.3.1. Phân hệ tài khoản

Phân hệ tài khoản phải hỗ trợ:

- Đăng ký bằng email và mật khẩu.

- Gửi email xác minh.
- Đăng nhập bằng email đã được xác minh.
- Đăng nhập hoặc tạo hồ sơ ứng dụng bằng Google.
- Đăng xuất.
- Quên mật khẩu.
- Đổi mật khẩu.
- Chỉnh sửa thông tin cá nhân.
- Cập nhật ảnh đại diện.
- Khôi phục hoặc làm mới trạng thái đăng nhập theo cơ chế hiện có.
- Khóa giao diện bằng sinh trắc học khi người dùng bật tính năng.

Các quy tắc nghiệp vụ quan trọng:

1. Client không được tự quyết định email đã xác minh.
2. Tài khoản bị khóa phải bị từ chối truy cập dù token còn hợp lệ về mặt chữ ký.
3. Đổi mật khẩu cần xử lý phiên cũ.
4. Google account trùng email với tài khoản legacy không được gộp âm thầm.
5. Người dùng không được chỉnh sửa hồ sơ của tài khoản khác bằng cách thay ID gửi lên.

Chức năng quên mật khẩu sử dụng luồng khôi phục phù hợp với phương thức xác thực. Đối với tài khoản Google, ứng dụng không được mô tả mình là nơi quản lý mật khẩu Google của người dùng.

3.3.2. Phân hệ chùa

Phân hệ chùa cung cấp danh mục địa điểm và nội dung tham khảo, gồm:

- Danh sách chùa.
- Tìm kiếm theo thông tin được hỗ trợ.
- Sắp xếp tên.
- Tìm chùa gần theo vị trí.
- Xem địa chỉ, giờ mở cửa, giới thiệu và lịch sử.
- Xem album nhiều ảnh.
- Mở chỉ đường.
- Lưu chùa yêu thích theo cơ chế cục bộ hiện có.

Album ảnh cần hỗ trợ nhiều ảnh cho một chùa thay vì chỉ một thumbnail. Khi một ảnh không tải được, giao diện phải có nội dung thay thế và không làm hỏng toàn bộ trang chi tiết.

Chức năng tìm chùa gần phải hoạt động có điều kiện: chỉ tính khoảng cách khi cả vị trí người dùng và tọa độ chùa hợp lệ. Nếu người dùng không cấp quyền, hệ thống vẫn cho phép tra cứu thông thường.

3.3.3. Phân hệ check-in NFC

Check-in là nghiệp vụ có tác động ghi dữ liệu nên cần đăng nhập.

Yêu cầu chính gồm:

- Kiểm tra khả năng NFC trên thiết bị.

- Đọc định danh và dữ liệu thẻ khi phù hợp.
- Gửi yêu cầu check-in đến backend.
- Xác minh thẻ đã đăng ký.
- Xác minh chùa liên kết còn hoạt động.
- Kiểm tra thông tin chùa gửi kèm không mâu thuẫn với đăng ký.
- Ghi nhận lần viếng.
- Cấp tem nếu chưa có.
- Trả kết quả rõ ràng cho người dùng.

Ứng dụng không được tự cấp tem chỉ vì đã đọc được một chuỗi UID. Quyết định cuối cùng nằm ở backend.

3.3.4. Phân hệ tem và lịch sử hành hương

Hai loại dữ liệu cần được phân biệt:

- **Lần viếng:** ghi nhận từng lần check-in.
- **Tem chùa:** đại diện cho việc người dùng đã từng check-in thành công tại một chùa.

Một người dùng có thể có nhiều lần viếng cùng một chùa nhưng chỉ có một tem tương ứng. Đây là lý do hệ thống sử dụng hai collection riêng.

Các chức năng gồm:

- Xem lịch sử viếng chùa.
- Xem thời điểm check-in.
- Xem bộ sưu tập tem.
- Xem số chùa đã đến, số lượt viếng và số tem.
- Làm mới dữ liệu sau check-in.
- Bảo đảm dữ liệu được truy vấn theo người dùng hiện tại.

3.3.5. Phân hệ kinh sách

Phân hệ kinh sách phục vụ nhu cầu đọc và tụng kinh trên thiết bị di động.

Các yêu cầu gồm:

- Xem danh sách kinh và chú.
- Tìm kiếm nội dung theo trường được hỗ trợ.
- Lọc theo loại.
- Đọc nội dung chi tiết.
- Trình bày văn bản dễ theo dõi.
- Bắt đầu hoặc dừng tự động cuộn.
- Điều chỉnh tốc độ cuộn.
- Thu gọn phần điều khiển để dành diện tích cho nội dung.

Tự động cuộn được thiết kế như một tiện ích hỗ trợ đọc. Khi người dùng chạm, kéo hoặc đến cuối nội dung, hệ thống phải xử lý trạng thái phù hợp để tránh cuộn mất kiểm soát.

3.3.6. Phân hệ Phật ngôn

Trang chủ hiển thị một câu Phật ngôn và cung cấp chức năng đổi câu.

Yêu cầu không chỉ là chọn ngẫu nhiên mà còn bao gồm:

- Tránh trả lại ngay câu hiện tại khi có lựa chọn khác.
- Hiển thị trạng thái đang tải và lỗi.
- Giữ nội dung đầy đủ.
- Hiển thị nguồn khi có.
- Giữ nhịp và dòng kệ khi có dữ liệu gốc đáng tin cậy.

Ở lần cập nhật gần nhất, việc khôi phục xuống dòng được thực hiện bằng đối chiếu với bản Kinh Pháp Cú đầy đủ trong repository. Chỉ những câu có nội dung khớp mới dùng định dạng gốc; các câu chưa khớp không bị tự động sửa lời.

3.3.7. Phân hệ nhân vật Phật giáo

Phân hệ tra cứu nhân vật bổ sung kiến thức liên quan đến các nội dung người dùng tiếp cận tại chùa.

Các chức năng gồm:

- Tìm kiếm theo tên.
- Lọc theo nhóm Phật, Bồ Tát, La Hán, Tổ sư và các nhóm khác.
- Xem tên gọi khác.
- Xem tên Sanskrit nếu có.
- Xem giới thiệu và tiểu sử.
- Xem biểu tượng, đặc điểm hình tướng.
- Xem ảnh đại diện hoặc nội dung thay thế khi thiếu ảnh.

Nhóm đã điều chỉnh cách trình bày trang chi tiết theo các khối nội dung và ảnh chữ nhật bo góc để phù hợp với yêu cầu giao diện.

3.3.8. Phân hệ lịch âm dương và sự kiện

Phân hệ lịch cần phân biệt hai nguồn dữ liệu:

1. Các ngày lễ Phật giáo được xây dựng từ dữ liệu nội dung của đồ án.
2. Sự kiện cá nhân do người dùng tạo.

Sự kiện cá nhân hỗ trợ các thông tin như tiêu đề, thời gian bắt đầu, kết thúc, địa điểm, mô tả, nhắc trước và lặp lại.

Hệ thống còn có:

- Hiển thị ngày âm và ngày dương.
- Xem lễ sắp tới.
- Xem chi tiết lễ.
- Thiết lập nhắc lịch.
- Đồng bộ lịch cá nhân với backend.
- Kiểm soát quyền sở hữu khi sửa hoặc xóa.

Các sự kiện lấy từ tài liệu Word phải được chuyển thành dữ liệu nghiệp vụ phù hợp, không phụ thuộc vào bố cục nguyên bản của tệp Word.

3.3.9. Phân hệ cộng đồng

Ứng dụng có liên kết đến nhóm Zalo cộng đồng:

```
https://zalo.me/g/xkofsw6frkf7qrigizmq
```

Chức năng hiện tại là mở cộng đồng bên ngoài. Đây chưa phải một phân hệ mạng xã hội nội bộ có bài đăng, bình luận, kiểm duyệt hoặc nhắn tin do backend của đồ án quản lý.

3.4. Yêu cầu phi chức năng

Bảng 3.2. Yêu cầu phi chức năng và định hướng đáp ứng

Nhóm	Yêu cầu	Cách đáp ứng hoặc tiêu chí đánh giá
Bảo mật	Chỉ chủ sở hữu truy cập dữ liệu cá nhân	Xác minh token và lọc theo người dùng
Toàn vẹn	Không cấp nhiều tem cùng chùa cho một người	Unique index và transaction
Hiệu năng	Giảm truy vấn lặp dữ liệu công khai	Redis cache có TTL
Chống lạm dụng	Hạn chế request quá nhanh	Rate limit và mã 429
Khả dụng	Thông báo lỗi rõ ràng	Trạng thái tải, rỗng, lỗi và thử lại
Khả năng phục hồi	Dịch vụ phụ gặp lỗi không gây hành vi mơ hồ	Chính sách lỗi riêng cho cache, rate limit, ảnh
Bảo trì	Dễ xác định nơi sửa logic	Tách services, controllers, models
Đa nền tảng	Tái sử dụng mã nguồn	Flutter và cấu hình native riêng
Kiểm thử	Có thể kiểm tra độc lập từng lớp	Unit, widget và integration test
Triển khai	Tái lập môi trường	Lockfile, Docker và CI
Chất lượng dữ liệu	Nội dung và ảnh liên kết đúng	Kiểm tra schema, đường dẫn và bản xem trước

Không nên đưa các số liệu như “phản hồi dưới 200 ms” hoặc “phục vụ hàng nghìn người đồng thời” vào phần kết quả nếu chưa có đo đạc. Những giá trị đó chỉ có thể là mục tiêu của bài kiểm thử hiệu năng trong tương lai.

3.5. Thiết kế kiến trúc tổng thể

Hệ thống được tổ chức theo mô hình sau:



3.5.1. Luồng đọc dữ liệu công khai

Ví dụ khi tải danh sách chùa:

1. Flutter gửi GET request.
2. Backend định tuyến đến chức năng tương ứng.
3. Nếu endpoint có cache và request phù hợp, middleware kiểm tra Redis.
4. Khi cache hit, phản hồi được trả về từ cache.
5. Khi cache miss, controller truy vấn MongoDB.
6. Backend trả JSON và có thể lưu kết quả vào Redis.
7. Client chuyển JSON thành model.
8. Giao diện hiển thị danh sách.

3.5.2. Luồng truy cập dữ liệu cá nhân

Ví dụ khi xem lịch sử hành hương:

1. Client lấy token hiện có.
2. Gửi request kèm header xác thực.
3. Middleware xác minh token.
4. Backend xác định user từ danh tính đã được kiểm tra.
5. Controller truy vấn lịch sử theo user đó.
6. Kết quả được trả về client.

Điểm quan trọng là controller không được tin một `userId` tùy ý do client gửi lên để quyết định quyền truy cập.

3.5.3. Luồng ảnh đại diện

1. Người dùng chọn ảnh.
2. Client gửi ảnh đến backend.
3. Backend xác thực người dùng.
4. Bộ phận xử lý upload tiếp nhận dữ liệu.
5. Backend tải ảnh lên nơi lưu trữ được cấu hình.
6. URL được cập nhật vào hồ sơ.

7. Client làm mới thông tin cá nhân và ảnh trên các màn hình liên quan.

3.6. Thiết kế cơ sở dữ liệu

Hệ thống sử dụng chín model nghiệp vụ chính. Tên model và tên collection vật lý có thể khác nhau do quy tắc đặt tên của Mongoose.

3.6.1. Model User

Trường	Vai trò
`_id`	Định danh người dùng ứng dụng
`firebaseUid`	Liên kết danh tính Firebase
`fullName`	Họ tên hiển thị
`email`	Email chuẩn hóa và duy nhất
`passwordHash`	Mật khẩu băm của tài khoản legacy
`avatar`	Địa chỉ ảnh đại diện
`role`	Vai trò `user` hoặc `admin`
`status`	Trạng thái `active` hoặc `blocked`
`tokenVersion`	Kiểm soát hiệu lực token legacy
`createdAt`, `updatedAt`	Thời gian tạo và cập nhật

`firebaseUid` có ràng buộc duy nhất và cho phép không tồn tại ở tài khoản legacy. `passwordHash` không được chọn mặc định khi truy vấn, giúp hạn chế việc vô tình đưa trường này vào phản hồi.

3.6.2. Model Temple

Nhóm trường	Nội dung
Định danh	`name`, `slug`, `otherNames`
Nội dung	`description`, `history`, `tradition`
Địa chỉ	`street`, `ward`, `city`, `country`
Tọa độ	`latitude`, `longitude`
Hình ảnh	`thumbnail`, `images`
Thời gian	`foundedYear`, giờ mở và đóng cửa
Liên hệ	Điện thoại, website, email
Hiển thị	`isFeatured`, `status`

Chỉ mục phục vụ tên, nội dung mô tả và địa chỉ thành phố. Cấu trúc tọa độ hiện tại là hai trường số, không nên mô tả thành truy vấn GeoJSON hoặc chỉ mục `2dsphere` khi schema chưa sử dụng.

3.6.3. Model NfcTag

Trường	Vai trò
`uid`	UID chuẩn hóa và duy nhất
`templeId`	Chùa được liên kết
`label`	Nhãn mô tả thẻ
`status`	Trạng thái hoạt động
Timestamps	Theo dõi tạo và cập nhật

Một chùa có thể có nhiều thẻ. Một thẻ được liên kết đến một chùa trong bản ghi hiện tại.

3.6.4. Model PilgrimageVisit

Trường	Vai trò
`userId`	Người thực hiện
`templeId`	Chùa được viếng
`nfcTagId`	Thẻ dùng để ghi nhận
`visitDate`	Thời điểm viếng
`note`	Ghi chú
Timestamps	Theo dõi bản ghi

Chỉ mục theo người dùng và thời gian giúp truy vấn lịch sử theo thứ tự mới nhất.

3.6.5. Model TempleStamp

Trường	Vai trò
`userId`	Chủ sở hữu tem
`templeId`	Chùa tương ứng
`firstVisitId`	Lần viếng đầu tiên dùng để cấp tem
`receivedAt`	Thời điểm nhận

Unique index trên cặp `userId` và `templeId` bảo đảm mỗi người chỉ có một tem cho mỗi chùa.

3.6.6. Model Scripture

Các trường chính gồm `title`, `slug`, `type`, `description`, `content`, `source`, `imageUrl`, `order`, `isFeatured` và `status`.

Trường `type` hiện sử dụng hai giá trị "Kinh" và "Chú". Nội dung chi tiết được lưu dưới dạng chuỗi, trong đó ký tự xuống dòng có ý nghĩa đối với cách trình bày.

3.6.7. Model Quote

Các trường chính gồm:

- `code`: mã câu.
- `content`: nội dung.
- `author`: tác giả nếu có.
- `source`: nguồn.
- `category`: nhóm nội dung.
- `isActive`: cho phép sử dụng hay không.

Việc có mã riêng giúp ứng dụng gửi tham số loại trừ câu hiện tại khi lấy câu tiếp theo.

3.6.8. Model CalendarEvent

Trường	Vai trò
`_id`	Chuỗi UUID
`userId`	Chủ sở hữu sự kiện
`title`	Tiêu đề
`startAt`, `endAt`	Thời gian bắt đầu và kết thúc
`usesLunarDate`	Sử dụng quy tắc ngày âm
`location`	Địa điểm
`description`	Nội dung
`reminderMinutes`	Số phút nhắc trước
`repeat`	Không lặp, ngày, tuần, tháng hoặc năm

Schema hiện lưu `startAt` và `endAt` dạng chuỗi để giữ cách biểu diễn thời gian mà phía Flutter sử dụng cho lịch địa phương và các sự kiện lặp. Vì vậy, việc chuyển đổi thời gian phải được xử lý nhất quán ở các lớp liên quan.

3.6.9. Model BuddhistFigure

Các trường chính gồm tên, slug, tên Sanskrit, tên gọi khác, danh mục, giới thiệu, tiểu sử, biểu tượng, hình tướng, thumbnail, album ảnh và trạng thái.

Danh mục được giới hạn bằng enum, giúp dữ liệu lọc trên client thống nhất với backend.

3.6.10. Quan hệ giữa các thực thể

```
User 1 ── N PilgrimageVisit N ── 1 Temple
User 1 ── N TempleStamp     N ── 1 Temple
Temple 1 ── N NfcTag
NfcTag 1 ── N PilgrimageVisit
User 1 ── N CalendarEvent
TempleStamp ── tham chiếu lần viếng đầu tiên
```

Kinh sách, Phật ngôn và nhân vật Phật giáo là các tập nội dung tra cứu độc lập tương đối với dữ liệu hành hương.

3.7. Đặc tả các luồng nghiệp vụ quan trọng

3.7.1. Đăng ký bằng email

Tiền điều kiện: Firebase được cấu hình và phương thức Email/Password đã bật.

Luồng chính:

1. Người dùng nhập thông tin.
2. Client kiểm tra dữ liệu ban đầu.
3. Firebase tạo danh tính.
4. Email xác minh được gửi.
5. Người dùng mở liên kết.
6. Ứng dụng lấy trạng thái xác thực mới.
7. ID token được gửi đến backend.
8. Backend xác minh token và email.
9. Backend tạo hoặc tìm hồ sơ ứng dụng phù hợp.
10. Người dùng được truy cập các chức năng cá nhân.

Ngoại lệ: email chưa xác minh, token hết hạn, dịch vụ chưa cấu hình, trùng email với hồ sơ cũ hoặc tài khoản bị khóa.

3.7.2. Đăng nhập Google

1. Người dùng chọn Google Sign-In.
2. Chọn tài khoản Google.
3. Client hoàn thành đăng nhập Firebase.
4. Lấy Firebase ID token.
5. Gửi token đến endpoint đăng nhập Firebase của backend.
6. Backend xác minh claims.
7. Tra cứu theo `firebaseUid`.
8. Tạo hồ sơ nếu chưa có và không xung đột email.
9. Trả thông tin phiên theo quy ước của ứng dụng.

Backend không cần biết mật khẩu Google của người dùng.

3.7.3. Đổi mật khẩu

Đối với tài khoản Firebase dùng mật khẩu, backend yêu cầu phương thức đăng nhập phù hợp và lần xác thực gần đây. Mã nguồn kiểm tra thời gian xác thực trong khoảng cho phép trước khi cập nhật mật khẩu.

Sau khi đổi, backend thu hồi refresh token và yêu cầu đăng nhập lại. Đối với legacy, phiên bản token được sử dụng để làm mất hiệu lực các token cũ.

3.7.4. Check-in và cấp tem

Tiền điều kiện: người dùng đã đăng nhập, thẻ và chùa hoạt động.

Luồng chính:

1. Client đọc thẻ.
2. Gửi UID và thông tin liên quan.
3. Middleware xác thực người dùng.
4. Backend chuẩn hóa UID.
5. Bắt đầu transaction.
6. Tra cứu thẻ.
7. Tra cứu chùa liên kết.
8. Kiểm tra slug hoặc temple ID nếu client gửi kèm.
9. Tạo lần viếng.
10. Kiểm tra tem hiện có.
11. Tạo tem nếu chưa có.
12. Tính số lượt viếng cần trả về.
13. Commit transaction.
14. Trả kết quả cho ứng dụng.

Nếu lỗi xảy ra trong transaction, các thay đổi chưa hoàn tất được rollback, tránh trạng thái ghi nhận không đồng bộ giữa lịch sử và tem.

3.7.5. Tạo sự kiện và nhắc lịch

1. Người dùng nhập sự kiện.
2. Client kiểm tra dữ liệu.
3. Backend xác thực và gắn chủ sở hữu.
4. Sự kiện được lưu.
5. Client cập nhật danh sách.
6. Bộ lập kế hoạch tạo thời điểm nhắc.
7. Thông báo được đăng ký trên thiết bị.
8. Khi thay đổi hoặc xóa sự kiện, kế hoạch nhắc được cập nhật.

3.7.6. Nhập dữ liệu chùa mới

Quy trình được tách thành:

```
Tiếp nhận dữ liệu
    → Chuẩn hóa
    → Kiểm tra đường dẫn ảnh và tọa độ
    → Tạo bản xem trước
    → Người thực hiện kiểm tra
    → Seed/cập nhật database khi được cho phép
```

Việc tách kiểm tra khỏi seed giúp giảm nguy cơ ghi đè nhầm dữ liệu đang chạy, đặc biệt khi dữ liệu liên quan đến thẻ, tem và lịch sử hành hương.

CHƯƠNG 4. HIỆN THỰC HỆ THỐNG, KIỂM THỬ VÀ TRIỂN KHAI

4.1. Tổ chức mã nguồn

4.1.1. Phía Flutter

Cấu trúc khái quát:

```
lib/
├── config/
├── models/
├── services/
├── widgets/
└── views/
    ├── startup/
    ├── trang_chu/
    ├── temples/
    ├── kinh_sach/
    ├── buddhist_figures/
    ├── nfc/
    ├── qr/
    ├── lich/
    ├── tai_khoan/
    └── tu_tap/
```

- `config` chứa cấu hình API và giao diện.
- `models` biểu diễn dữ liệu nghiệp vụ.
- `services` xử lý API, lưu trữ cục bộ và các tích hợp.
- `widgets` chứa thành phần tái sử dụng.
- `views` tổ chức màn hình theo phân hệ.

Các cấu hình đặc thù nền tảng được đặt trong `android/` và `ios/`, không trộn vào toàn bộ mã giao diện Dart.

4.1.2. Phía backend

```
backend/
├── src/
│   ├── config/
│   ├── controllers/
│   ├── middleware/
│   ├── models/
│   ├── routes/
│   ├── seed/
│   └── utils/
├── data/
├── img/
├── test/
├── Dockerfile
├── compose.test.yaml
├── package.json
└── package-lock.json
```

Dữ liệu nội dung được tách khỏi logic xử lý, giúp nhóm cập nhật danh mục chùa hoặc kinh sách mà không cần sửa trực tiếp controller.

4.2. Hiện thực các chức năng cốt lõi

4.2.1. Xác thực và quản lý hồ sơ

Các thành phần chính gồm dịch vụ xác thực phía Flutter, lớp tích hợp Firebase, controller xác thực Firebase và middleware backend.

Middleware xử lý hai nhánh:

- Token có tiền tố Firebase.
- JWT legacy.

Đối với Firebase, hệ thống xác minh token, tìm tài khoản theo UID và kiểm tra trạng thái. Đối với JWT, hệ thống xác minh chữ ký, tìm người dùng và đối chiếu `tokenVersion`.

Controller Firebase không tự động liên kết với tài khoản legacy có cùng email. Đây là một quyết định bảo mật quan trọng vì lịch sử hành hương và lịch cá nhân có thể thuộc một hồ sơ đã tồn tại.

4.2.2. Quên mật khẩu và xử lý phiên

Giao diện quên mật khẩu được tách thành màn hình riêng. Người dùng nhập email và nhận hướng dẫn khôi phục thông qua phương thức xác thực được sử dụng.

Đổi mật khẩu là một quy trình khác với quên mật khẩu. Người dùng đang đăng nhập vẫn có thể bị yêu cầu xác thực lại để chứng minh quyền kiểm soát tài khoản gần thời điểm thao tác.

Sau khi đổi mật khẩu, ứng dụng cần chuyển về trạng thái yêu cầu đăng nhập lại thay vì tiếp tục sử dụng token đã không còn phù hợp.

4.2.3. Hồ sơ và ảnh đại diện

Ảnh được chọn bằng `image_picker`, gửi đến backend và lưu theo cấu hình avatar storage. Khi sử dụng Cloudinary, URL kết quả được ghi vào hồ sơ.

Việc cập nhật avatar phải phản ánh ở cả trang cá nhân lẫn các thành phần khác dùng chung hồ sơ, chẳng hạn góc tài khoản trên trang chủ. Vấn đề từng gặp là một màn hình đã có ảnh mới nhưng màn hình khác vẫn giữ trạng thái cũ; điều này cho thấy cần thống nhất cách làm mới dữ liệu người dùng.

4.2.4. Chùa, album ảnh và GPS

Model chùa có `thumbnail` và `images`, cho phép tách ảnh dùng trong danh sách với album trên trang chi tiết.

Bộ dữ liệu mới bổ sung tọa độ và nhiều ảnh cho từng chùa. Quá trình nhập gần nhất đã kiểm tra tính tồn tại của đường dẫn, tính hợp lệ của tọa độ và khả năng đối chiếu tên chùa. Những tên gần giống nhau vẫn phải được phân biệt chính xác, tránh gán ảnh hoặc tọa độ cho sai địa điểm.

Chức năng GPS sử dụng tọa độ để sắp xếp. Trong khi đó, thao tác chỉ đường được chuyển đến bản đồ ngoài bằng `url_launcher`.

4.2.5. Check-in NFC, lịch sử và tem

Controller NFC sử dụng `session.withTransaction` để nhóm các thao tác ghi.

Một lần check-in hợp lệ tạo một bản ghi `PilgrimageVisit`. Nếu người dùng chưa có tem cho chùa đó, hệ thống tạo thêm `TempleStamp`; nếu đã có, hệ thống chỉ ghi nhận lượt viếng mới.

Phản hồi chứa thông tin chùa, thẻ, lượt viếng và tem, bao gồm cờ cho biết tem có mới hay không. Nhờ đó, client có thể hiển thị thông báo phù hợp mà không suy đoán.

Một điểm cần tiếp tục đánh giá là chống gửi lặp yêu cầu. Unique index đã bảo vệ tem nhưng không có nghĩa mọi request lặp đều bị hợp nhất thành một lần viếng. Cơ chế idempotency hoặc khoảng chờ theo nghiệp vụ là hướng bổ sung phù hợp.

4.2.6. Đọc kinh và tự động cuộn

Trang đọc kinh sử dụng bộ điều khiển cuộn để theo dõi vị trí hiện tại và giới hạn nội dung.

Phần điều khiển tự cuộn đã được điều chỉnh theo yêu cầu:

- Đặt ở phía trên.
- Chỉ hiện phần điều chỉnh tốc độ khi người dùng mở.
- Giảm diện tích chiếm dụng của thanh công cụ.
- Cho phép dừng và tiếp tục.
- Xử lý khi đến cuối nội dung.

Cách tổ chức này phù hợp với màn hình điện thoại, nơi nội dung đọc dài cần được ưu tiên hơn các nút điều khiển.

4.2.7. Phật ngôn và khôi phục dòng kệ

Dữ liệu Phật ngôn từng có dạng một chuỗi văn xuôi dù nội dung gốc là kệ nhiều dòng. Nếu chỉ căn giữa chuỗi đó, giao diện vẫn không thể khôi phục nhịp đọc.

Lần chỉnh sửa gần nhất bổ sung bảng đối chiếu dòng kệ từ nội dung Kinh Pháp Cú đầy đủ. Logic hiển thị thực hiện:

1. Giữ nguyên nếu nội dung đã có xuống dòng.
2. Tìm bản kệ theo mã câu.
3. So sánh nội dung sau chuẩn hóa khoảng trắng.
4. Chỉ dùng bản nhiều dòng khi nội dung thực sự khớp.
5. Giữ nguyên nội dung API nếu không khớp.

Đã đối chiếu được 330 câu trong tổng số 350 câu của bộ dữ liệu đang sử dụng. Hai mươi câu còn lại được giữ nguyên để tránh sửa sai nội dung.

Ví dụ câu kệ 342 được trình bày:

```
Người bị ái buộc ràng,
Vùng vẫy và hoảng sợ,
Như thỏ bị sa lưới.
Họ sanh ái trói buộc,
Chịu khổ đau dài dài.
```

Thay đổi này tác động đến cách hiển thị trên client, không yêu cầu tự động seed lại database.

4.2.8. Lịch cá nhân, ngày lễ và thông báo

Sự kiện cá nhân được lưu ở backend theo chủ sở hữu. Các ngày lễ được xử lý bởi dịch vụ lễ và logic chuyển đổi ngày.

Bộ lập kế hoạch nhắc lịch tổng hợp lễ sắp tới cùng sự kiện cá nhân, xét thời gian nhắc trước và quy tắc lặp. Các thời điểm đã qua không được lên lịch lại như thông báo tương lai.

Đây là phần cần kiểm thử kỹ ở các tình huống chuyển ngày, đổi múi giờ, sự kiện lặp và sử dụng ngày âm. Một thông báo được đăng ký thành công cũng chưa bảo đảm người dùng sẽ luôn nhận đúng lúc trên mọi thiết bị, vì hệ điều hành còn có chính sách quyền và tiết kiệm pin.

4.2.9. Redis cache và giới hạn request

Cache sử dụng khóa có tiền tố riêng và giá trị băm của URL. Chỉ phản hồi thành công, có cấu trúc phù hợp và kích thước nằm trong giới hạn mới được lưu.

Rate limit sử dụng Lua để tăng bộ đếm và đặt thời hạn theo cách nguyên tử. Điều này tránh tình trạng bộ đếm được tạo nhưng không có thời gian hết hạn do các thao tác rời rạc.

Phản hồi rate limit có mã 429 và `Retry-After`. Khi hệ thống giới hạn đang bật nhưng không thể thực thi, phản hồi 503 thể hiện rõ dịch vụ tạm thời chưa sẵn sàng.

4.3. Phương pháp kiểm thử

4.3.1. Kiểm tra tĩnh

Lệnh:

```
flutter analyze --no-pub
```

được sử dụng để phát hiện các vấn đề như sai kiểu, sử dụng API không đúng, biến không dùng và một số vi phạm quy tắc mã nguồn.

Kiểm tra tĩnh không chứng minh luồng nghiệp vụ hoạt động hoàn toàn đúng, nhưng giúp loại bỏ một nhóm lỗi trước khi chạy ứng dụng.

4.3.2. Unit test và widget test phía Flutter

Unit test tập trung vào hàm, model và service. Widget test kiểm tra hành vi giao diện trong môi trường giả lập của Flutter.

Ví dụ với Phật ngôn:

- Service gửi mã câu cần loại trừ.
- Service xử lý phản hồi lỗi.
- Giao diện đổi câu sau khi nhấn nút.
- Nguồn trích dẫn hiển thị đúng.
- Câu 342 được khôi phục thành năm dòng.
- Nội dung đã bị chỉnh sửa không bị bảng đối chiếu ghi đè.
- Nội dung đã có xuống dòng được giữ nguyên.

HTTP client giả lập cho phép kiểm thử mà không phụ thuộc backend production.

4.3.3. Kiểm thử tích hợp backend

Backend sử dụng Node Test Runner. Các tệp kiểm thử hiện bao gồm:

- API nghiệp vụ.
- Firebase claims.
- Redis.
- Lưu ảnh đại diện.
- Nội dung Phật ngôn.

Môi trường Compose khởi tạo MongoDB 7 với replica set `rs0`. Hai thư mục dữ liệu database được đặt trên `tmpfs` để phục vụ môi trường thử nghiệm tạm thời.

Replica set cần thiết vì luồng check-in sử dụng transaction. Nếu chỉ kiểm thử bằng MongoDB standalone, kết quả có thể không phản ánh đúng môi trường cần thiết cho nghiệp vụ này.

4.3.4. Ma trận kiểm thử đề xuất

Phân hệ	Trường hợp chính	Kết quả mong đợi
Đăng ký	Email chưa xác minh	Không được truy cập nghiệp vụ yêu cầu xác minh
Google	Người dùng mới	Tạo hồ sơ đúng UID
Tài khoản	Email trùng legacy	Không tự gộp hồ sơ
Token	Hết hạn hoặc bị thu hồi	Yêu cầu đăng nhập lại
Mật khẩu	Đổi thành công	Phiên cũ không tiếp tục dùng như trước
Hồ sơ	Tải avatar	Cập nhật URL và giao diện
NFC	Thẻ hợp lệ	Ghi lần viếng, cấp tem nếu mới
NFC	Thẻ lạ hoặc vô hiệu	Không ghi dữ liệu
NFC	Slug không khớp thẻ	Từ chối yêu cầu
Tem	Quét lại cùng chùa	Không tạo thêm tem trùng
Lịch	Sửa sự kiện người khác	Bị từ chối
GPS	Không cấp quyền	Vẫn dùng được tìm kiếm thông thường
GPS	Chùa thiếu tọa độ	Không làm lỗi danh sách
Nhắc lịch	Tắt nhắc	Không tạo lịch nhắc tương ứng
Kinh	Đến cuối khi tự cuộn	Dừng phù hợp
Phật ngôn	Nội dung thay đổi	Không bị thay bằng bản cũ
Redis	Cache lỗi	Đọc từ nguồn chính
Rate limit	Vượt ngưỡng	Trả 429
Rate limit	Redis lỗi khi bật giới hạn	Trả 503 theo thiết kế

4.3.5. Kết quả đã ghi nhận và giới hạn

Hạng mục	Kết quả có căn cứ
Phân tích mã nhánh `locfrontend`	Đã chạy và không phát hiện lỗi
Bộ kiểm thử Flutter của bản `locfrontend` được kiểm tra trước chỉnh sửa dòng kệ	57 bài kiểm thử đạt
Kiểm thử riêng sau thay đổi định dạng Phật ngôn	4 bài kiểm thử đạt
Phân tích ba tệp Dart thuộc thay đổi định dạng	Không phát hiện lỗi
APK release `locfrontend`, build 19	Build thành công, khoảng 74,4 MB
Chạy iOS	Người thực hiện xác nhận thành công
NFC thực tế	Người thực hiện đã xác nhận quét thành công ở giai đoạn trước
Backend trên snapshot mới nhất	Chưa chạy lại toàn bộ trong lần biên soạn này
Kiểm thử tải nhiều người dùng	Chưa có số liệu đo
Kiểm thử toàn bộ chức năng iOS	Chưa đủ căn cứ xác nhận hoàn tất

Không cộng 57 và 4 thành 61 vì bốn bài kiểm thử là một lần chạy tập trung sau thay đổi, có phần chồng lấp với bộ kiểm thử trước.

APK build 19 cũng được tạo **trước** lần chỉnh sửa định dạng câu kệ mới nhất, nên cần build lại nếu muốn kiểm tra thay đổi đó trên điện thoại.

4.4. Quy trình CI/CD

4.4.1. Bốn job CI

Job	Môi trường	Nhiệm vụ
Local CD safety tests	Windows	Kiểm tra script CD cục bộ
Flutter analyze and test	Ubuntu	Cài dependency, phân tích mã, chạy test, lưu coverage
Backend integration tests	Ubuntu	Chạy Compose kiểm thử và lưu log
Android test APK	Ubuntu	Build APK debug sau khi các job phụ thuộc đạt

Job APK phụ thuộc vào Flutter và backend. Nếu một trong hai thất bại, bước build APK có thể bị bỏ qua.

Các artifact được lưu giúp nhóm tải kết quả và kiểm tra lỗi mà không phải chạy lại toàn bộ môi trường ngay trên máy cá nhân.

4.4.2. Workflow xuất bản Docker image

Workflow CD chỉ chạy trong các điều kiện phù hợp:

- CI hoàn tất thành công.
- Sự kiện gốc là push.
- Nhánh thuộc phạm vi `main`.
- Repository nguồn đúng repository của dự án.

Sau đó, workflow checkout đúng SHA đã kiểm thử và xuất bản image có tag:

```
ghcr.io/<repository>/backend:sha-<commit-sha>
```

Cách gắn tag này giúp xác định image tương ứng với mã nguồn nào, thay vì chỉ dựa trên một tag thay đổi liên tục.

4.4.3. Triển khai Render

`render.yaml` khai báo backend dạng Docker Web Service và health check tại:

```
/api/health
```

Các thông tin nhạy cảm được cấu hình qua biến môi trường hoặc Secret Files. Firebase Admin sử dụng service account ở backend; không đưa private key vào ứng dụng mobile.

Sau khi triển khai, cần kiểm tra cả health endpoint lẫn các nghiệp vụ phụ thuộc dịch vụ ngoài. Health endpoint hoạt động chưa đủ để khẳng định Google login, Cloudinary và Redis đều đúng cấu hình.

4.5. Đóng gói và phát hành ứng dụng

4.5.1. Android release

Quy trình gồm:

1. Chọn commit cần phát hành.
2. Kiểm tra dependency và mã nguồn.
3. Chạy kiểm thử.
4. Cung cấp cấu hình API/Firebase.
5. Cấu hình keystore.
6. Build APK release.
7. Kiểm tra file đầu ra.
8. Cài thử trên điện thoại.
9. Phân phối cho người dùng.

Ví dụ:

```
flutter build apk --release --dart-define-from-file=firebase-config.json --build-number=20
```

Số build cần tăng phù hợp với bản đang phân phối. Việc cập nhật backend qua CD không tự thay thế APK đã cài trên máy người dùng; những thay đổi giao diện hoặc logic Dart vẫn cần phát hành bản ứng dụng mới.

4.5.2. Điều chỉnh môi trường build trên máy cá nhân

Trong quá trình thực hiện, lỗi build Android từng xuất phát từ thiếu bộ nhớ native của JVM. Cấu hình Gradle được điều chỉnh để phù hợp máy 16 GB RAM:

- Heap tối đa 3 GB.
- Metaspace tối đa 1 GB.
- Code cache 256 MB.
- Tối đa hai worker.
- Tắt build song song.

Sau điều chỉnh, APK được build thành công. Trường hợp này cho thấy cần phân biệt lỗi tài nguyên môi trường với lỗi mã nguồn hoặc xung đột dependency.

4.5.3. iOS và bài học về tương thích thư viện

Quá trình build iOS từng gặp xung đột dependency giữa Firebase iOS SDK và Google Sign-In ở thư viện dùng chung GTMSessionFetcher.

Biện pháp xử lý là giữ nhóm phiên bản FlutterFire tương thích đã được kiểm tra, thay vì nâng cấp riêng lẻ một package lên major mới.

Bản iOS được build bằng môi trường macOS từ xa, sau đó ký/cài bằng quy trình sideload. Người thực hiện đã chạy thành công, nhưng đây không phải quy trình phát hành App Store và không đồng nghĩa tất cả capability đều khả dụng.

4.6. Các giới hạn cần ghi nhận khi bàn giao

1. **Dữ liệu mới chưa đồng nghĩa dữ liệu production:** bộ 27 chùa đang chờ kiểm tra trước khi seed.
2. **Bản định dạng Phật ngôn mới chưa có trong APK build 19:** cần đóng gói lại.
3. **Redis chưa quản lý phiên:** chức năng đang có là cache và rate limit.
4. **Thông báo là thông báo cục bộ:** chưa nên mô tả thành hệ thống push từ backend.
5. **Mục yêu thích có thành phần lưu trên thiết bị:** chưa được xem là đồng bộ đa thiết bị theo tài khoản.
6. **Sinh trắc học là lớp khóa tại thiết bị:** không thay thế xác thực API.
7. **NFC chưa chứng minh sự hiện diện tuyệt đối:** cần giải pháp mạnh hơn nếu dùng trong nghiệp vụ chống gian lận.
8. **Danh sách thẻ NFC có endpoint công khai trong mã nguồn được kiểm tra:** cần đánh giá lại phạm vi công khai nếu nâng yêu cầu bảo mật.
9. **Render là phụ thuộc triển khai bên ngoài:** cần đánh giá tính sẵn sàng theo gói thực tế thay vì khẳng định chạy liên tục không gián đoạn.
10. **Lưu token cục bộ cần tiếp tục cải thiện:** việc sử dụng `shared_preferences` trong dịch vụ xác thực không tương đương kho bí mật được phần cứng bảo vệ.
11. **Chưa có bằng chứng kiểm thử tải lớn:** khả năng phục vụ nhiều người phải được đo trước khi công bố.
12. **Nội dung Phật giáo cần được thẩm định:** kiểm tra phần mềm không thay thế việc kiểm chứng nguồn, bản dịch và tính chính xác của dữ liệu.

4.7. Đánh giá kết quả hiện thực

Đồ án đã hình thành một hệ thống gồm ứng dụng di động, API, cơ sở dữ liệu và các dịch vụ hỗ trợ triển khai. Các nhóm chức năng chính đã có mã nguồn hiện thực: tài khoản, xác thực email và Google, hồ sơ, chùa, album ảnh, tìm chùa gần, kinh sách, tự cuộn, Phật ngôn, nhân vật Phật giáo, lịch, nhắc lịch, check-in NFC, tem và lịch sử hành hương.

Điểm nổi bật của hệ thống là kết hợp dữ liệu nội dung với nghiệp vụ có trạng thái và tích hợp phần cứng. Việc check-in không chỉ tạo thông báo trên màn hình mà còn liên quan đến đăng ký thẻ, xác thực người dùng, transaction, lịch sử và ràng buộc tem duy nhất.

Quá trình phát triển cũng đã xử lý các vấn đề tích hợp thực tế như địa chỉ API khác nhau giữa máy giả lập và điện thoại, lưu trữ ảnh khi triển khai đám mây, vô hiệu hóa phiên sau đổi mật khẩu, xung đột dependency iOS và giới hạn bộ nhớ khi build Android.

Mức độ hoàn thiện cần được đánh giá theo từng phạm vi đã kiểm chứng. Các kết quả build và kiểm thử tự động là bằng chứng quan trọng, trong khi kiểm thử thiết bị, chất lượng nội dung, khả năng chịu tải và tính sẵn sàng production vẫn cần được bổ sung trước khi mở rộng sử dụng ngoài phạm vi đồ án.